

Rings: A peer-to-peer network for sovereign age

Ryan J. Kung
ryankung@ieee.org

Created: February 8, 2023
Source: RingsNetwork/whitepaper repository
Timestamp: 2023-02-08T13:27:55Z
Last updated: July 4, 2026

Abstract

Rings Network is a decentralized peer-to-peer network for the sovereign age. This paper describes a target architecture in which browsers and native devices participate as first-class nodes, identities are addressed as DIDs on a Chord overlay, resources and services are mapped into the same identifier space, and applications compose privacy-preserving protocols such as encrypted messaging, offline mailboxes, hidden service descriptors, ranking, and sidecar ordering. The model is formalized as protocol objects over the circular carrier $\mathbb{Z}/2^{160}\mathbb{Z}$, with placement functors into the live Chord topology and Kleisli morphisms for effectful protocol transitions. This paper presents the ideal Rings model while updating directions that have been superseded: Chord identifiers are treated as an additive circular space for routing and placement, cryptographic groups and fields are separate protocol carriers, and message relay, ranking, and sequencing are specified as Rings protocols rather than inherited wholesale from unrelated systems.

1 Introduction and Motivation

Centralized systems have long posed significant privacy and security risks to Internet users, as evidenced by numerous studies [1–4]. The emergence of decentralized technologies has therefore attracted considerable interest as a means of mitigating these risks, while moving towards a "sovereign age" characterized by increased user control and data ownership.

The Rings Network represents a decentralized peer-to-peer alternative that aims to address these privacy and security challenges, offering a secure and private means of conducting online interactions. The objective of this paper is to introduce the Rings Net-

work and demonstrate its potential for mitigating the risks posed by centralized systems, thus advancing the development of a sovereign age.

The technical specifications and architecture of the Rings Network will be thoroughly analyzed and its potential impact on personal privacy and the internet will be explored. These findings will contribute to the ongoing discussions about the future of the internet and the role of decentralized technologies in enhancing privacy, security, and control for users.

This paper aims to make a meaningful contribution to the discourse surrounding decentralized solutions such as the Rings network and their ability to advance towards a more secure, private and sovereign

Internet.

1.1 Browser Native

The Rings Network is characterized as a browser-native network at the protocol level: a browser tab can own an identity, establish peer-to-peer transport, participate in routing, store replicated state within its capability profile, and host applications without depending on an application server. Native and browser environments expose different operating-system capabilities, but the ideal model gives both the same overlay authority and isolates environment-specific IO behind adapters.

An important aspect of the Rings Network is its communication layer, which is based on WebRTC [5] and WebAssembly [6]. These technologies support browser-to-browser peer-to-peer networking and allow data traffic to avoid a central application server once peers have enough connectivity information to establish a WebRTC data channel.

WebRTC, as a standard for real-time communication on the internet, offers a smooth user experience for the Rings Network. Meanwhile, WebAssembly, a low-level binary format for web-based applications, provides the necessary performance for the network. These technologies are widely supported by contemporary browsers, making the Rings Network accessible to a vast user base.

Moreover, the integration of WebRTC and WebAssembly allows decentralized applications to run on top of the same overlay. The target application space includes secure file sharing, decentralized web publishing, private mailboxes, verifiable computation, relay/tunnel services, and market or exchange protocols that can be expressed as namespace-scoped applications.

1.2 Proof Based DID

Decentralized Identifiers (DIDs) in Rings are protocol identities located on the Chord identifier circle.

A participant proves control of a DID through an account proof and a delegated session proof. Rings does not require a W3C DID-document resolver in the core overlay path; DID resolution is a routing and proof problem inside the overlay.

The target DID proof system is pluggable but not arbitrary. Each proof family must define a verifier, a domain-separated transcript, DID derivation, signature encoding, replay bounds, and whether the public key is recoverable from the signature or must be carried explicitly. This keeps Rings compatible with multiple cryptographic ecosystems without losing a uniform proof boundary.

Incorporating DID proofs into the Rings network facilitates the development of secure and trusted methods for identifying and authenticating network participants. This, in turn, promotes the establishment of secure and confidential communications and data exchange based on trusted relationships between parties.

1.3 Lookup Protocol and Encryption

The Rings Network leverages the Chord algorithm [7] for overlay lookup. The network is organized as a circular identifier space, and each node maintains successor, predecessor, and finger-table state to route messages and storage operations toward a destination DID or resource identifier.

In conjunction with the lookup protocol, the Rings Network uses signed message payloads and end-to-end encryption. Peers exchange signed public-key handshakes, then carry ordered ciphertext frames inside signed message envelopes. Secret sharing, private storage, and verifiable computation use their own cryptographic fields or groups; they must not assume that the Chord identifier space itself is a finite field.

These lookup protocol and encryption capabilities provide a secure and efficient method of communication and data sharing in the Rings Network, surpassing the privacy and security offerings of traditional centralized networks.

1.4 Data and Service Provision

Rings maps resources and service records into the same 160-bit identifier space used by the Chord overlay. Virtual DIDs represent data, mailboxes, service descriptors, subrings, and application-specific rendezvous points. A virtual DID has no account proof of its own; its authenticity comes from signed entry operations, encrypted payloads, capability descriptors, and the application protocol that defines how the resource may be read or updated.

2 Related Work

In the field of decentralized networks, Rings Network is not the sole option available. There are several other decentralized networks, such as Tor, GnuNet, Nym, and IPFS, that aim to provide users with privacy, security, and censorship resistance. In this section, we will undertake a comparative analysis of Rings Network with other peer-to-peer (P2P) networks.

In P2P networks, Distributed Hash Tables (DHTs [8]) are commonly utilized. Many prominent decentralized networks, such as Edonkey [9], Ethereum [10], IPFS [11], and Libp2p [12], have adopted the Kademlia [13] algorithm as their DHT solution. Rings Network instead employs the Chord protocol for structured routing. This gives Rings a deterministic ring model for successor, predecessor, and finger-table reasoning. Broadcast and gossip performance should be evaluated as Rings protocol behavior rather than inferred solely from the choice of DHT.

On the other hand, Rings Network belongs to the category of structured P2P networks, while Nym, Bitcoin, and BitTorrent networks are considered unstructured P2P networks. Structured P2P networks, such as Rings Network, have a well-defined topology and routing mechanism, while unstructured P2P networks rely on nodes to discover and connect with each other in a random manner. This structured architecture grants Rings Network a higher degree of scalability, efficiency, and robustness compared to

unstructured P2P networks.

In terms of P2P communication protocols, the Chord protocol is employed by Rings Network, whereas IPFS and Ethereum utilize Kademlia-derived systems. Rings' design goal is to use the structured ring as the base for lookup, relay, storage placement, and service rendezvous, while keeping privacy-sensitive network addresses out of application-level identifiers whenever possible.

In contrast, Tor network and Nym network are not structured P2P networks but rather relay networks that are decentralized but not P2P. Rings takes a different path by making the structured overlay itself the routing and service-discovery substrate. This reduces dependence on fixed relay infrastructure, while traffic-analysis and metadata-resistance claims still depend on the selected relay, ranking, mailbox, and cover-traffic protocols.

Finally, it is worth noting that Rings Network is a browser-native P2P network, a unique feature that sets it apart from other existing network implementations. Rings Network can be executed on browsers, mobile devices, or PCs, thus making it a true P2P network that provides every individual with complete control over their data sovereignty.

3 The Rings Network Architecture

The Rings Network architecture is not a direct re-statement of the TCP/IP reference model [14]. It is a sovereign-network architecture organized around six cooperating layers: identity, transport, overlay, extension runtime, protocol, and application.

- Identity Layer: DID addressing, account proofs, delegated sessions, and virtual-resource identities.
- Transport Layer: browser and native peer-to-peer data channels, NAT traversal, and transport capability profiles.
- Overlay Layer: the Chord DHT maintains suc-

cessor, predecessor, finger-table, storage, and message-routing state.

- Extension Runtime Layer: namespace-scoped protocols are modeled as state transitions whose effects are interpreted by constrained shells.
- Protocol Layer: relay, mailbox, ranking, side-car sequencing, service discovery, private storage, and verifiable-computation protocols.
- Application Layer: decentralized web, secure messaging, hidden services, proof applications, and user-defined sovereign applications.

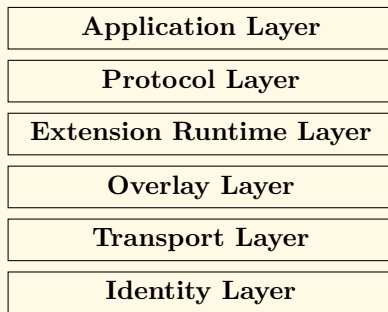


Figure 1: Layers of Rings Network

3.1 Identity Layer

The design goal of Rings Network is to make the same overlay and identity model available from multiple runtime surfaces. Native nodes, mobile nodes, embedded nodes, and browser nodes should share DID addressing, delegated sessions, Chord routing, and message semantics. Each runtime supplies the IO adapters available in that environment, so equality means equal protocol authority rather than identical local resources.

The identity layer owns DID derivation, account-proof verification, delegated sessions, and virtual-resource identities. It names who or what is allowed to sign a protocol operation; it does not decide transport reachability, overlay placement, or application-specific authorization policy.

3.2 Transport Layer

The transport layer uses WebRTC data channels and native transport adapters to give browser and non-browser nodes peer-to-peer connectivity. The browser-based approach means that users do not need to install additional software or plugins to participate in the network. WebRTC allows the network to target low latency and high throughput for real-time communication and data transfer, while native adapters can expose capabilities unavailable to a browser tab.

WebRTC, or Web Real-Time Communication, provides browsers and mobile apps with real-time communication capabilities through simple APIs. With WebRTC, users can easily send audio, video, and data streams directly between browsers, without the need for any plug-ins or extra software. At the same time, the Rings Network has some special optimizations for the WebRTC handshakes process. Assuming Node A and Node B want to create a WebRTC connection, they would need to exchange a minimum of three messages with each other:

- Node A gathers a list of ICE candidates and generates an SDP offer. The SDP offer includes information about the media streams and network addresses that Node A is willing to receive or transmit.
- Node A sends the SDP offer to Node B.
- Node B receives the SDP offer and starts gathering its own list of ICE candidates.
- Node B generates an SDP answer in response to the SDP offer received from Node A. The SDP answer includes information about the media streams and network addresses that Node B is willing to receive or transmit.
- Node B sends the SDP answer and its list of ICE candidates back to Node A.
- Node A receives the SDP answer and the list of ICE candidates from Node B.
- Node A and Node B use the ICE candidates and SDP information to establish a connection and establish a peer-to-peer communication channel.

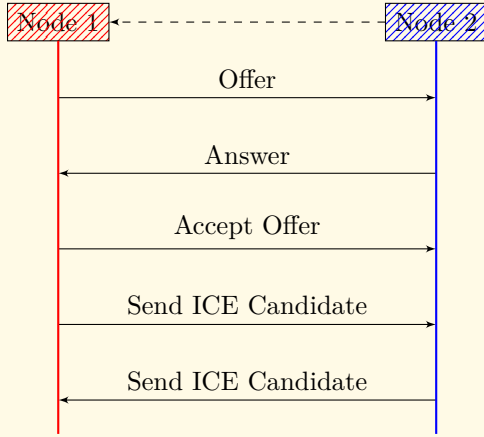


Figure 2: Exchange SDP

By optimizing the WebRTC handshakes process, the Rings Network reduces the latency and overhead associated with establishing WebRTC connections. This results in a more efficient and reliable communication experience for users.

- Gather ICE candidates and create an SDP offer, which is sent to the other party.
- The recipient accepts the offer and generates an SDP answer, which is sent back along with its own list of ICE candidates.
- At this point, the connection is established and the peer-to-peer communication channel is ready for use.

At its core, WebRTC is based on the exchange of Session Description Protocol (SDP) between browsers. The SDP is used to negotiate the communication settings and exchange cryptographic keys, allowing for secure communication between browsers. The process of exchanging SDP and establishing a secure connection is known as the WebRTC handshake.

In Rings Network, the WebRTC handshake is optimized by generating both the ICE Candidates and Offer simultaneously and exchanging them in a single Request-Response transaction. This significantly improves the efficiency of the handshake between nodes.

Once the WebRTC handshake is complete,

browsers can send and receive data streams directly, with low latency and high bandwidth. This makes WebRTC ideal for real-time communication applications, such as video conferencing, online gaming, and live streaming.

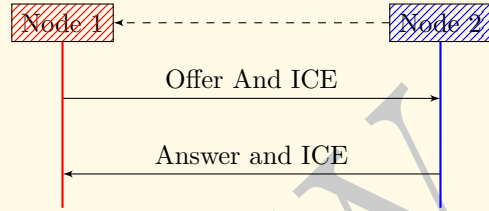


Figure 3: Exchange SDP

3.3 Overlay Layer

The Rings Network is a structured peer-to-peer network that incorporates a distributed hash table (DHT) to facilitate efficient and scalable lookups. The Chord algorithm is utilized to implement lookup within the DHT, thereby enabling message routing and replicated storage operations in a peer-to-peer setting. Availability depends on transport connectivity, stabilization progress, redundancy settings, and churn; the DHT gives the protocol a structured basis for routing rather than a standalone availability guarantee.

Each node in the Rings Network is assigned a range of keys and maintains a finger table containing information about other nodes in the network. This allows for quick determination of the node responsible for a given key and facilitates the forwarding of queries to the appropriate node. The nodes and keys within the network are arranged in an identifier ring, which has at most 2^m nodes, ranging from 0 to $2^m - 1$. The identifier ring provides a convenient method for mapping keys to nodes, and each node in the identifier ring has a successor and a predecessor. The successor of a node is the next node in the identifier ring in a clockwise direction, while the predecessor is the previous node in a counter-clockwise direction.

To maintain the robustness of the network in

the event of node failures or departures, each node records a segment of the ring adjacent to it, including the preceding and following nodes. This results in a high probability that a node will be able to accurately locate its successor or predecessor, even in networks that experience a high failure rate. The information regarding the closest successor or predecessor of a node is stored in the finger table maintained by each node.

The Rings Network uses Chord-style finger routing to reduce query times and improve lookup efficiency. When a node seeks to look up a key, it forwards the query to the closest known predecessor or successor of the key, as determined by the information stored in its finger table and successor list, until the responsible node is found. This approach reduces the number of nodes that must be traversed to locate the responsible node, providing a scalable solution for data storage and retrieval within a peer-to-peer network.

In this way, the Chord algorithm provides a scalable way to store and retrieve data in a peer-to-peer network. The use of the hash function and finger table allows efficient routing of queries and updates, while redundancy, stabilization, and repair policies provide the basis for availability and robustness against node failures.

In the DHT, as illustrated in Figure 4, all nodes have the capability to perceive their own predecessor and successor through the join and stabilization algorithms. The predecessor of a node is determined as the node with the largest identifier that precedes the identifier of the current node in the identifier space. Conversely, the successor of a node is defined as the node with the smallest identifier that follows the identifier of the current node in the identifier space. These two nodes, the predecessor and the successor, are considered the immediate neighbor nodes of a particular node within the network.

3.4 Extension Runtime Layer

The extension runtime layer is the interpreter boundary for namespace-scoped protocol logic. Protocol

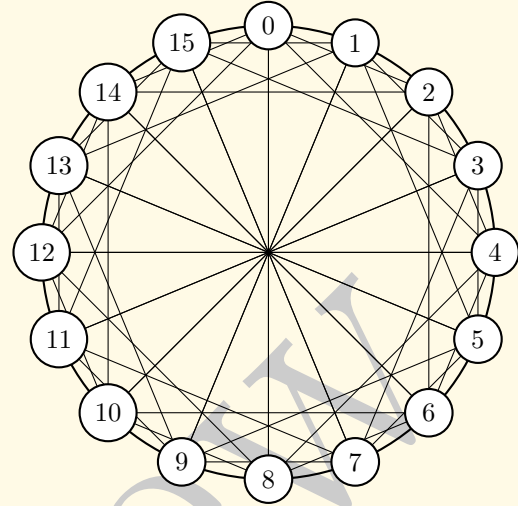


Figure 4: Chord algorithm, lookup protocol

handlers should be modeled as pure state transitions that return effect descriptions, while the runtime supplies capability-limited host calls for storage, transport, timers, and cryptographic verification. This keeps extension behavior replayable and testable without giving each extension direct ambient access to the node process.

WebAssembly, or Wasm, is a low-level binary format for executing code in web browsers. It provides a way for developers to write high-performance code in languages such as C, C++, and Rust, and then run that code in a browser with native performance. WebAssembly is designed to be fast and efficient, with a compact binary format that is easy to parse and execute. It uses a low-level instruction set, which makes it well-suited for demanding tasks such as cryptography, compression, and image processing.

The Rings Network provides multiple options for interacting with WebAssembly (Wasm) modules:

- Build browser applications using wasm-based frameworks such as Yew and Trunk.
- Load the exported JavaScript/Wasm module directly from web applications.

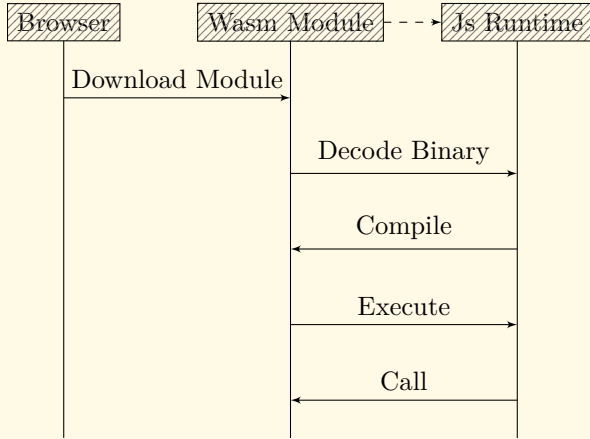


Figure 5: How wasm module works

- Use a browser provider or extension workflow, where applications interact with a Rings provider surface and register namespace-scoped handlers.

3.5 Protocol Layer

In the protocol layer, the central design concept revolves around the utilization of a Decentralized Identifier (DID). A DID is a 160-bit Chord identifier in a space with 2^{160} points, represented as an element of the additive cyclic group $\mathbb{Z}/2^{160}\mathbb{Z}$. Addition, subtraction, and negation model clockwise offsets, biased ordering, finger targets, and affine replica placement. DID multiplication is outside the Rings routing model: cryptographic multiplication belongs to the fields or groups used by specific cryptographic protocols, not to Chord identifiers themselves.

However, elements in a circular identifier space cannot be compared by a single global less-than relation that is meaningful for every observer. Algorithm 1 addresses this limitation. The algorithm selects a fixed point C and compares A and B by their clockwise offsets from C . In this manner, ordering is always relative to the observer or operation that chose the fixed point.

Algorithm 1 Bias Order

Input: Elements A and B , and fixed point C . **Output:** Result of comparison between A and B

```

1: calculate  $A - C$  and  $B - C$ 
2: if  $A - C < B - C$  then
3:   return  $A < B$ 
4: else
5:   if  $A - C > B - C$  then
6:     return  $B < A$ 
7:   else
8:     return  $A = B$ 
  
```

The DID, as an element of the Chord identifier space, also provides a placement target for resources and storage entries. Resource identifiers are derived into the same 160-bit space so the same lookup and placement logic can route storage, mailbox, service, and subring operations.

The DID architecture also encompasses signed Session and Message Relay modules. Sessions delegate a temporary session keypair from an account proof, and message payloads carry signed routing metadata so handlers can process system messages, storage messages, encrypted frames, mailbox operations, and namespace-scoped extension messages.

3.6 Application Layer

The nucleus of Rings Network is similar to the Actor Model [15]: inbound messages are decoded into typed protocol messages and handled by explicit handler traits. The extension runtime described above separates protocol logic into a pure state transition and an interpreter shell. This allows built-in and user-defined protocols to run behind namespace-scoped capabilities without adding a global command bus.

Rings Network provides a comprehensive set of interfaces for application-layer messages, which can be accessed through Remote Procedure Calls (RPC) or Foreign Function Interface (FFI). This implies that application developers have the option to not only control Rings Network through Rust or WebAssembly (Wasm), but can also access it via RPC us-

ing lighter weight programming languages such as Python or JavaScript. Furthermore, embedded developers can integrate Rings Network into microcontrollers through the use of FFI.

Additionally, applications can store data in the Distributed Hash Table (DHT), look up services, and register namespace-scoped protocols. In the ideal model, peer quality and global ranking provide admission and routing signals, while local counters remain the lowest-level evidence source.

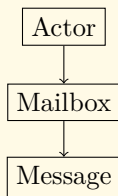


Figure 6: Actor Model

Several application classes motivate Rings Network: decentralized chat, dWeb publishing, relay/tunnel services, browser proof applications, private mailbox delivery, and hidden-service style rendezvous. Concrete demos may change over time; the paper should describe the application class and protocol requirement rather than depending on historical demo names.

4 Algebraic Network

4.1 Classic Chord

Rings builds on the Chord distributed hash table rather than inventing a new overlay from first principles. In Chord, nodes and keys are mapped by consistent hashing into an m -bit circular identifier space, usually written as $[0, 2^m)$. The owner of a key is its successor: the first live node encountered when moving clockwise around the identifier circle. Each node stores at least its successor and predecessor, and it may maintain a finger table whose i -th entry points near $n + 2^i$ to accelerate lookup [7].

Chord lookup is a routing procedure over this circular order. If the target key lies between the local node and its successor, the successor is returned. Otherwise the query is forwarded to the closest preceding known node, usually obtained from the finger table, until a node can answer the successor relation. Periodic maintenance then repairs the local view: stabilization asks the current successor for topology information, notification lets a successor learn a better predecessor, and finger repair refreshes long-distance routing hints.

This base model gives Rings its first algebraic object: a circular identifier carrier with relative order, successor, predecessor, and finger relations. It also defines the separation between correctness and performance. The successor/predecessor structure is the safety-critical ring-maintenance state, while fingers are an optimization that should improve hop count without becoming the proof root for membership correctness.

4.2 Correct Chord as Maintenance Model

The original Chord paper is not enough as a formal maintenance model under churn. Zave's Correct Chord work shows that the original ring-maintenance protocol was not correct under its stated operating assumptions, then gives corrected operations, initialization constraints, and an inductive invariant for preserving reachability in Chord identifier spaces [16]. In particular, Correct Chord makes successor lists and a stable base of at least $r + 1$ members central to the correctness argument, where r is the successor-list length.

Rings adopts Correct Chord as the maintenance reference before adding Rings-specific semantics. The relevant transition family is join, rectify, and stabilize. Join introduces a node through a known peer and constructs a bounded successor list. Rectify processes a predecessor notification and accepts it only when it is a better predecessor candidate. Stabilize refines the successor list from the current successor's topology. Finger maintenance remains a routing op-

timization, and storage repair is a Rings extension layered on top of Chord placement.

4.3 Rings as an Algebraic Object

The Rings model is not merely "Chord plus applications." It is a family of protocol objects indexed over one circular carrier, with effects interpreted through a monadic runtime boundary. We use standard category-theoretic language for the shape of the construction [17], and monadic language for effectful protocol composition [18].

Definition 1 (Carrier and circular order). *Let*

$$M = 2^{160}, \quad R = \mathbb{Z}/M\mathbb{Z}.$$

For an observer $a \in R$, define clockwise distance and interval membership by

$$\delta_a(x) = (x - a) \bmod M,$$

$$(a, b]_R = \{x \in R : 0 < \delta_a(x) \wedge \delta_a(x) \leq \delta_a(b)\}.$$

The carrier operation used by the overlay is addition in R . Multiplication in R is not a routing operation.

For a finite live set $N \subset R$, the mathematical owner of a key k is

$$\text{owner}_N(k) = \arg \min_{n \in N} \delta_k(n).$$

The bounded successor sequence of node n is

$$\text{Succ}_N^r(n) = \text{take}_r(\text{sort}_{\delta_n}(N \setminus \{n\})).$$

Correct Chord gives the maintenance discipline for making local successor-list state converge toward these mathematical functions; Rings adds typed identities, resources, storage, sessions, and extension protocols on top of them.

Definition 2 (Rings object). *A Rings overlay state over N is the tuple*

$$\mathcal{R}_N = (R, N, \text{owner}_N, \text{pred}, \text{Succ}^r, F, E)$$

where F is the sparse finger relation and E is the replicated entry store indexed by VIDs. The safety root is $(\text{pred}, \text{Succ}^r)$; F is a performance witness, and E is a protocol-indexed payload algebra.

Rings' main categorical move is to put every routable thing in the slice category $\mathbf{Set}/R$. A DID namespace, a mailbox namespace, a service-descriptor namespace, and a subring namespace are all sets equipped with a derivation map into R :

$$H_\nu : X_\nu \longrightarrow R.$$

DIDs are account-controlled elements of such a namespace. VIDs are resource-controlled or protocol-derived elements. Both are objects over the same carrier, not elements of the same cryptographic group.

For a fixed live topology N , placement is the composite

$$P_\nu^N = \text{owner}_N \circ H_\nu : X_\nu \longrightarrow N.$$

Viewing X_ν and N as discrete categories, P_ν^N is the placement functor for namespace ν . If a namespace morphism $f : X_\nu \rightarrow X_\mu$ preserves derivation,

$$H_\mu \circ f = H_\nu,$$

then placement is natural:

$$P_\mu^N \circ f = P_\nu^N.$$

This is the precise claim behind "DIDs, resources, mailboxes, service descriptors, and subrings share the same ring." They share placement by a commuting diagram over R ; they do not share authorization, payload semantics, or cryptographic carrier laws.

Law 1 (Carrier separation). *Overlay placement may use only*

$$0, \quad +, \quad -, \quad \delta_a, \quad (a, b]_R, \quad \text{owner}_N.$$

Any protocol requiring multiplication, inversion, scalar multiplication, polynomial interpolation, pairings, or subgroup checks must name a separate carrier such as a finite field $\mathbb{F}_q$ or elliptic-curve group $\mathbb{G}$. The only bridge from bytes or proofs into R is an explicit, domain-separated derivation map H_ν .

Law 2 (Rotation equivariance). *For every offset $t \in R$, define $T_t(x) = x + t$ and $N + t = \{n + t \mid n \in N\}$. Rings placement is independent of the absolute zero point:*

$$\text{owner}_{N+t}(k + t) = \text{owner}_N(k) + t.$$

Thus a diagram, storage policy, or test that depends only on circular offsets can be translated around the ring without changing its algebraic content.

For an action alphabet A , let A^* be the free monoid of finite action lists with identity ϵ and concatenation $\cdot$. The Rings effect monad is

$$\mathsf{T}_A(X) = X \times A^*,$$

with unit and bind

$$\eta(x) = (x, \epsilon),$$

$$(x, \alpha) \gg f = \text{let } f(x) = (y, \beta) \text{ in } (y, \alpha \cdot \beta).$$

Typed errors add a sum component, $\mathsf{E} + \mathsf{T}_A(X)$, but do not change the effect-composition law.

Definition 3 (Protocol object). *A Rings protocol namespace is a tuple*

$$\mathcal{P}_\nu = (X_\nu, S_\nu, I_\nu, O_\nu, A_\nu, H_\nu, \tau_\nu)$$

where X_ν is the descriptor space, S_ν is protocol state, I_ν is input, O_ν is output, A_ν is the action alphabet, $H_\nu : X_\nu \rightarrow R$ is placement, and

$$\tau_\nu : S_\nu \times I_\nu \rightarrow \mathsf{T}_{A_\nu}(S_\nu \times O_\nu)$$

is the transition. A morphism of protocols preserves placement and maps transitions to transitions; in other words, protocol refactoring must commute with H_ν and with τ_ν .

4.4 Rings Extension: DID Ring and Carrier Boundaries

On top of Chord, Rings replaces opaque node keys with protocol DIDs and VIDs. DID ownership is proven through account verification algorithms. Recoverable signature schemes can identify an account by DID/address, while non-recoverable schemes carry explicit public keys and derive DIDs from domain-separated public-key transcripts. This verifier interface is the stable boundary; new proof systems must implement it rather than bypassing session verification.

In Rings Network, the representation of Decentralized Identifiers (DIDs) is achieved through a fixed-width Chord identifier space. A DID is represented by a 160-bit value, and Chord interval comparisons are relative to an observer. Rings uses biased ordering to compare positions from a chosen reference point, and additive offsets for finger targets and redundant placement. This is additive cyclic-group arithmetic over $\mathbb{Z}/2^{160}\mathbb{Z}$, not protocol-level finite-field multiplication.

The term ring is therefore used carefully. A DID belongs to the Chord identifier ring because the identifier space wraps around modulo 2^{160} . Algebraically, the protocol depends on the additive abelian group structure of this carrier: zero is the local reference point, addition and subtraction model clockwise movement, and negation models movement in the opposite direction. Rings intentionally does not treat DID multiplication as a protocol operation. If a cryptographic protocol needs multiplication, inversion, or a finite field, it must use a separate scalar-field or curve-group carrier with its own laws.

The generation of resource identifiers for data and service records also uses the same identifier space. The DHT structure in Rings Network implements join, lookup, stabilization, storage sync, and storage repair operations over this shared Chord model.

4.5 Rings Extension: State Algebra

Rings expresses the Chord layer as a pure state transition system interpreted by a mutable node runtime. For a finite live set N and a local node n , Rings defines clockwise distance as:

$$\text{dist}_n(x) = (x - n) \bmod 2^{160}$$

The local state carried by a node is modeled as

$$S_n = (p_n, L_n, F_n, E_n)$$

where p_n is predecessor, L_n is a bounded successor sequence, F_n is a sparse finger partial function, and E_n is local replicated entry evidence. The mathematical target state is

$$p_n = \text{pred}_N(n), \quad L_n = \text{Succ}_N^r(n).$$

A finger slot is a partial witness for the Chord target:

$$target_i(n) = n + 2^i,$$

$$finger_i(n) = owner_N(target_i(n)).$$

Rings uses a sparse no-wrap finger table, so $F_n(i) = \perp$ is a valid routing-state fact when the known topology cannot witness the slot. A present finger is a performance morphism toward the owner relation, not an additional membership proof.

For storage and mailbox state, each VID v has a payload algebra E_v . The default replicated-entry law is a join semilattice:

$$(E_v, \sqsubseteq, \sqcup, \perp)$$

with commutativity, associativity, idempotence, and bottom identity. If a namespace cannot use semilattice merge, it must define an explicit conflict relation and finalization rule instead of inheriting DHT replication as if it were consensus.

This formalization keeps the academic Chord story intact, then names Rings' deviations explicitly. Rings keeps Correct Chord's successor-list and predecessor-centered safety model, but maps every participant, storage entry, mailbox, service descriptor, subring anchor, and application rendezvous point into the same DID/VID carrier. Rings also uses sparse no-wrap fingers in its current model, so high-index finger absence is a routing-state fact, not a Chord theorem.

4.6 Rings Extension: Monadic Effect Boundary

Rings also models side effects algebraically. A DHT transition should not directly perform network IO. It is a Kleisli morphism

$$S \longrightarrow \mathsf{T}_A(S \times O)$$

that returns a new state, an output, and a finite action list. Typical actions are find-successor, query-successor-list, try-connect, notify, sync-storage, and no-op. The message layer lowers protocol actions

into the coproduct of core effects: payload relay, fresh message send, connection management, storage placement, and timer scheduling.

Composition is Kleisli composition:

$$\tau_2 \star \tau_1 = \lambda s. \tau_1(s) \gg \tau_2.$$

Operationally, this means protocol composition concatenates action evidence rather than executing it. A runtime interpreter is a monoid homomorphism

$$I_A : A^* \rightarrow IO$$

such that $I_A(\epsilon)$ is no-op and $I_A(\alpha \cdot \beta)$ executes $I_A(\alpha)$ before $I_A(\beta)$. Browser, native, and embedded runtimes may choose different interpreters; the protocol law remains the same as long as they interpret the same action alphabet.

The target architecture therefore does not require every runtime to expose a generic Monad trait, but it does require the same separation of pure state transition, effect value, and interpreter. This keeps protocol laws testable without opening real sockets, while allowing runtime-specific IO adapters.

4.7 Rings Extension: Formal Verification Boundary

The Rings formal-methods style starts from the Chord and Correct Chord literature, then writes the Rings state machine in TLA-like state variables and operators. The state variables are the DID carrier $R = \mathbb{Z}/2^{160}\mathbb{Z}$, the live set N , predecessor map p , successor sequence L , sparse finger table F , replicated entry state E , and pending action list Q . The operators are join, rectify, stabilize, finger repair, storage sync, storage repair, and lookup.

The central safety predicate is that converged local state is a fixpoint of the mathematical topology:

$$\begin{aligned} \text{Inv}_{ring}(N, S) &\equiv \forall n \in N. p_n = \text{pred}_N(n) \\ &\quad \wedge L_n = \text{Succ}_N^r(n). \end{aligned}$$

Lookup safety is owner preservation:

$$\text{Lookup}(S, k) \Downarrow \text{owner}_N(k)$$

where $\Downarrow$ means that repeated Kleisli interpretation of remote lookup actions reaches the same owner as the mathematical function, under the modeled delivery assumptions.

Replica placement is a namespace-indexed invariant:

$$\text{Inv}_{\text{place}} \equiv \forall \nu, x. \text{replicas}(H_\nu(x)) \subseteq \text{Succ}_N^r(P_\nu^N(x)).$$

Effect transparency is the boundary invariant:

$$\pi_S(\tau(s, i)) = s' \quad \text{is independent of } I_A,$$

meaning that pure transition results do not depend on the runtime interpreter chosen for browser, native, or embedded IO.

The verification boundary is intentionally scoped. Deterministic convergence checks show that, for representative finite layouts, the pure Chord operators materialize the expected safety fixpoint. Stateright models exhaust finite message interleavings for selected subprotocols such as predecessor notification, a small discovery abstraction, storage CRDT merge, and hand-off cleanup. Trace replay checks the real multi-hop lookup implementation on converged rings against the mathematical successor relation. These artifacts support the paper's Chord and DID algebra, but they do not claim a full proof of asynchronous liveness, arbitrary network fairness, Byzantine behavior, or every production runtime schedule.

4.8 Network DID

In a network topology, all the nodes form a ring structure. Rings uses Chord-derived join, lookup, and stabilization algorithms for the ring network. The algorithms below are protocol-level sketches of the algebraic transitions; a production node emits action values that are interpreted by the message and transport layers.

When a participant joins the network, Algorithm 2 models the join as reply-driven local transitions. The node uses an existing peer as an entry point, returns an action asking the overlay to locate its successor,

Algorithm 2 DHT Join Transitions

```

1: function BEGINJOIN( $n, \text{known}$ )
2:    $\text{pred}[n] \leftarrow \text{none}$ 
3:    $a \leftarrow \text{FindSuccessor}(n.\text{did})$ 
4:   return  $\text{Remote}(\text{known}, a)$ 
5: function INSTALLSUCCESSOR( $n, s$ )
6:    $\text{succ}[n] \leftarrow \text{take}(r, [s])$ 
7:   return  $\text{Remote}(s, \text{QuerySuccList}(s))$ 
8: function INSTALLSUCCLIST( $n, s, \text{list}$ )
9:    $\text{succ}[n] \leftarrow \text{take}(r, s :: \text{list})$ 
10:  if  $\text{head}(\text{succ}[n]) \neq \text{none}$  then
11:     $h \leftarrow \text{head}(\text{succ}[n])$ 
12:     $\text{notify} \leftarrow \text{Notify}(h, n.\text{did})$ 
13:     $\text{sync} \leftarrow \text{SyncStorage}(h)$ 
14:    return  $\text{Multi}(\text{notify}, \text{sync})$ 
15:  else
16:    return  $\text{Noop}$ 

```

installs the successor returned by that action, then asks the successor for its successor list. Once the reply arrives, the node initializes a bounded successor sequence and returns follow-up actions for synchronization and notification. No join transition directly calls a remote peer or mutates remote state; it describes the local state update and the messages that the effect interpreter must deliver.

The **join** algorithm is therefore a local state transition plus an effect value. Awareness of the new node spreads through notify, stabilize, finger repair, and storage repair. Finger tables are routing indexes rather than authoritative membership maps.

Algorithm 3 locates the successor of a DID or VID. If the local successor sequence can answer the interval query, the transition returns a local owner. Otherwise it returns a remote action naming the next hop and the requested key. The $\text{next} = \text{none}$ branch is not an ownership claim: after an interval miss it is unreachable in a converged ring, and under partial local state it returns a repair action so the interpreter refreshes successor evidence before retrying. On a converged ring, repeatedly interpreting the remote action resolves to the mathematical successor of the target. The $O(\log N)$ routing expectation depends on

Algorithm 3 DHT Lookup Algorithm

```
1: function LOOKUP(key, node)
2:    $s \leftarrow \text{head}(\text{succ}[\text{node}])$ 
3:   if  $s = \text{none}$  then
4:     return Repair(QuerySuccList(node))
5:   if  $\text{key} \in (\text{node.did}, s]$  then
6:     return Local(s)
7:    $\text{next} \leftarrow \text{closest\_preceding\_node}(\text{node}, \text{key})$ 
8:   if  $\text{next} = \text{none}$  then
9:     return Repair(QuerySuccList(s))
10:  else
11:     $a \leftarrow \text{FindSuccessor}(\text{key})$ 
12:    return Remote(next, a)
```

sufficiently populated finger tables; correctness still falls back to the successor sequence.

Algorithm 4 DHT Stabilization

```
1: procedure STABILIZE(n, topo)
2:    $c \leftarrow \text{topo.pred}$ 
3:    $s \leftarrow \text{head}(\text{succ}[n])$ 
4:   if  $c \neq \text{none}$  and  $c \in (n.\text{did}, s)$  then
5:      $\text{succ}[n] \leftarrow \text{insert}(\text{succ}[n], c)$ 
6:     emit FindSuccessor(c)
7:    $\text{succ}[n] \leftarrow \text{take}(r, \text{merge}(\text{succ}[n], \text{topo.succ}))$ 
8:   if  $\text{head}(\text{succ}[n]) \neq \text{none}$  then
9:     emit Notify(head(succ[n]), n.did)
```

The **stabilization** algorithm is a monotone refinement step over known topology. It accepts a better successor candidate when the current successor reports a predecessor closer to the local node, merges bounded successor-list evidence, and emits a notification to the new head successor. Finger repair is a separate optimization pass that uses the same biased-distance order. Stabilization improves routing efficiency, storage placement, and hand-off safety, but the formal claim is scoped to convergence toward the Chord safety fixpoint under the modeled assumptions.

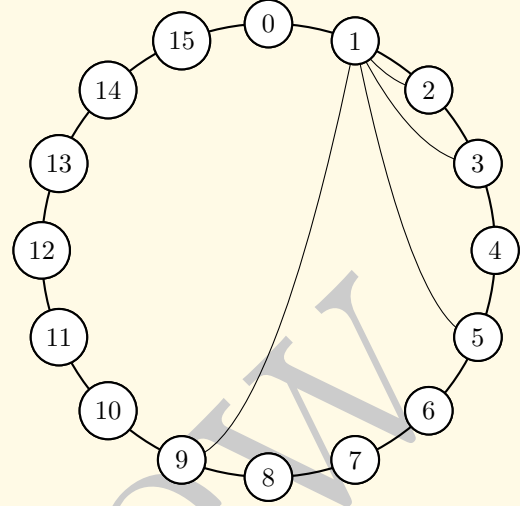


Figure 7: Chord algorithm, lookup protocol

4.9 Virtual DID

In Rings Network, Virtual DID (VID) serves as a network identifier that resembles a DID in format but does not represent an account-controlled node. A VID is a placement key for a resource, mailbox, service descriptor, subring, or application-specific state object. A VID has no private key and no DID proof; authorization comes from the operation applied to the entry and from signatures or capabilities carried in the entry payload.

For a resource descriptor x , a protocol derives a VID with a domain-separated hash, for example $H(\text{rings.resource} \parallel \text{namespace} \parallel x)$. Domain separation is required so that a mailbox, a service descriptor, and a storage chunk with the same user-visible name do not collide semantically even if they map into the same 160-bit identifier space.

Data storage is the foundation for advanced data storage operations. When a node stores data, it derives a VID and uses Chord lookup plus configured redundancy to place entries on responsible nodes. Large data is split into chunks; each chunk has its own VID, and the root entry stores a signed man-

ifest containing chunk identifiers, sizes, hashes, encryption metadata, and recovery policy. Storage capacity is an adapter and policy concern, not a fixed global protocol constant.

Mailbox provides asynchronous delivery for DIDs that are offline or unreachable. A mailbox root can be derived as $H(\text{rings.mailbox}||\text{DID}||\text{epoch})$, where the epoch limits scanning and storage growth. Senders append encrypted envelopes to the mailbox entry. Each envelope contains the recipient DID, sender session proof or anonymous credential, ciphertext, expiration, replay nonce, and optional postage or ranking evidence. The recipient retrieves mailbox entries by proving control of the DID, decrypts envelopes locally, and acknowledges or garbage-collects messages according to the mailbox policy. This design gives offline delivery without requiring direct interaction, but it does not hide the existence of a mailbox access pattern unless combined with cover traffic or private retrieval.

Hidden services are service descriptors stored under VIDs rather than ordinary DNS names. A descriptor contains a service namespace, rendezvous policy, relay or introduction points, capability requirements, public keys, expiration, and ranking/admission constraints. Clients look up the descriptor, verify its signature, choose an introduction path, and establish an encrypted application session through the overlay. Rings should claim service pseudonymity and server-location privacy only to the extent provided by the descriptor, relay-path selection, and traffic-analysis defenses.

Sub Ring is a scoped overlay whose membership and routing state are anchored by a VID. The subring name or policy derives the subring VID, while members join by submitting signed membership operations. The DHT-persisted state stores the membership log or a compact joined state; participants keep local finger tables using the subring's fixed point and membership set. Convergence requires that join operations are idempotent and mergeable, that conflicting membership updates have deterministic resolution, and that stale members expire or are removed by signed policy operations.

5 Traffic

In Rings Network, the abstraction of traffic is divided into two approaches: Hop-by-Hop and End-to-End. Hop-by-Hop refers to a communication architecture where data is transmitted from one device to another through multiple intermediate systems in between. On the other hand, End-to-End (E2E) refers to a communication architecture where data is transmitted directly from the sender's device to the recipient's device without passing through any intermediate systems.

5.1 Hop by Hop

In Rings Network, a structured peer-to-peer network, accounts prove DID ownership with account signature algorithms, then delegate a time-bounded session keypair. Each hop carries signed message payloads, routing metadata, and typed messages. The overlay infers a next hop from the local DHT when a direct transport connection to the final destination is not available.

The message relay and chunking layers borrow concepts from MSRP-style message splitting, but Rings should define its own relay semantics instead of claiming MSRP compliance. A Rings relay frame contains the path state needed by the overlay, a destination DID or VID, a namespace, a signed session proof, and either an application payload or a report. Reports acknowledge delivery, failure, forwarding refusal, or storage placement, giving applications enough information to retry or choose a different path.

5.2 End to End

Rings Network places significant emphasis on End-to-End Encryption as a means to ensure privacy and security. The target E2E protocol uses public keys that are proven to match the signed sender or recipient DID. RSA is not part of the Rings target unless a later design explicitly reintroduces it; the privacy layer should focus on elliptic-curve encryption, signed

envelopes, and cryptographic carriers that can support private computation.

Given an elliptic-curve cyclic group $\mathbb{G}$ of order q with generator g , a private key $x \in \mathbb{Z}_q$, and public key $h = g^x$, Algorithm 5 summarizes the ElGamal operation:

Algorithm 5 ElGamal Encryption and Decryption

```

1: Input: Message  $m$ , private key  $x \in \mathbb{Z}_q$ , public
   key  $h = g^x$ , generator  $g$ 
2: Output: Encrypted message  $(c_1, c_2)$ 
3: procedure ENCRYPTION
4:   Choose a random integer  $k \in \mathbb{Z}_q$ 
5:   Compute  $c_1 = g^k$ 
6:   Compute  $c_2 = m \cdot h^k$ 
7:   return  $(c_1, c_2)$ 
8: procedure DECRYPTION
9:   Compute  $m = c_2 \cdot (c_1)^{-x}$ 
10:  return  $m$ 

```

An encrypted stream divides byte payloads into ordered frames. Each frame carries a stream identifier, sender public key, monotonic sequence, final marker, and ciphertext blocks. The frame is then carried in a signed message envelope, which supplies sender authentication, public-key ownership, frame integrity, and truncation detection.

Secret sharing is a separate privacy-layer primitive. The secret is first encoded over an appropriate finite field, for example with a threshold polynomial. Shares are then encrypted to participant keys and placed on the overlay by VID. The Chord identifier space decides where shares live; it is not the field in which the threshold scheme is computed. Recovery requires at least the threshold number of valid shares plus the corresponding decryption authority.

5.3 Chunking

When the size of a message exceeds the negotiated transport message size, Rings splits the payload into ordered chunks and reassembles the original payload once all chunks arrive. The safe baseline is whole-

message buffering with bounded reassembly: the receiver accepts out-of-order chunks, ignores duplicate positions, drops partial messages by TTL, and enforces per-message and total memory limits. A future streaming mode must specify interruption, partial-delivery, and backpressure semantics explicitly.

Algorithm 6 End-to-End Chunking Algorithm

```

1: procedure E2E CHUNKING
2:   Input: data, chunk size
3:   Output: chunks
4:   chunks  $\leftarrow []$ 
5:   total-chunks  $\leftarrow \text{ceil}(\text{len}(\text{data})/\text{chunk-size})$ 
6:   for  $i$  in range(total-chunks) do
7:     chunk  $\leftarrow \text{data}[i*\text{chunk-size} : (i+1)*\text{chunk-size}]$ 
8:     append chunk to chunks
9:   return chunks

```

Algorithm 6 shows the sender-side shape of whole-message chunking. A robust receiver also validates timestamps, per-chunk size, total chunk count, total buffered cost, and completed-message tombstones.

6 Sequencer and Ranking

A sequencer in a distributed system is a component responsible for determining the order in which messages are delivered or transactions are executed. Rings should not embed a global consensus system into the base overlay; doing so would turn the network into a blockchain rather than a sovereign peer-to-peer substrate. Instead, Rings uses local protocol ordering where possible and sidecar witnesses where a stronger ordering proof is required.

6.1 Sidecar Sequencer

The Sidecar Sequencer is a witness protocol, not a mandatory base-layer consensus engine. A message that requires ordering carries a witness reference such as a block hash, timestamped commitment, signed

log root, or application-specific ordering certificate. The verifier checks three facts:

- the witness existed before or after a claimed bound;
- the message commitment is bound to that witness;
- the application accepts the witness source and its finality rule.

Sidecar sequencing therefore gives a partial order unless all compared messages use the same finality domain or the application defines a cross-domain comparison rule. If Message A is witnessed by Bitcoin and Message B by Solana, Rings cannot infer a universal total order from those witnesses alone. The application must either accept a partial order, choose a canonical witness domain, or define a deterministic tie-breaker.

6.2 Byzantine Fault Boundary

Rings does not solve Byzantine fault tolerance for every application at the overlay layer. It delegates ordering finality to the selected sidecar witness and verifies that the witness is correctly attached to a Rings message. This keeps the overlay small while allowing applications to select different security assumptions: a blockchain, a threshold signature committee, a transparency log, or a local Lamport-style clock for weaker ordering needs.

6.3 Ranking Protocol

The target Ranking Protocol turns peer quality into a signed, challengeable, and rate-limited signal. A ranking event contains:

- the subject DID and rater DID;
- the measured behavior, such as connection success, delivery success, storage availability, latency bucket, or protocol violation;
- evidence, such as signed reports, failed challenge transcripts, or availability probes;

- an epoch, expiration, and anti-replay nonce;
- the rater’s stake, credential, or trust weight.

Scores are aggregated by epoch with robust statistics such as median or trimmed mean so that outliers do not dominate. The ranking result is not a moral identity score; it is an operational routing and admission signal. A node may use ranking to choose relay paths, reject storage writes, select mailbox replicas, throttle spam, or require stronger payment/proof from low-quality peers.

Sybil resistance requires more than peer voting. The protocol must combine admission cost, proof-of-work or stake, social or issuer credentials, freeze periods for new rank claims, and challenge/slashing rules for false evidence. A low-ranked node should still be able to recover by producing fresh good evidence, while a malicious high-ranked node should lose score when challenged with signed failure evidence.

7 Performance

We will discuss the performance of Rings Network from two aspects: Latency, Reliability.

7.1 Latency

For Rings Network, latency mainly comes from transport establishment, lookup/routing through the Chord overlay, relay path length, sidecar witness verification, and application protocol work such as encryption or proof verification. The Chord reference data below is useful for reasoning about expected lookup behavior, but it is not a substitute for Rings-owned measurements under browser, native, NAT, relay, and churn conditions.

According to the cited Chord simulation results, when the number of nodes increases from 10 to 100K, the path length only increases by about 3 times. The left side of Figure 8 shows this scenario, and the right side is the PDF (probability density function) of the path length. Rings should validate this expectation

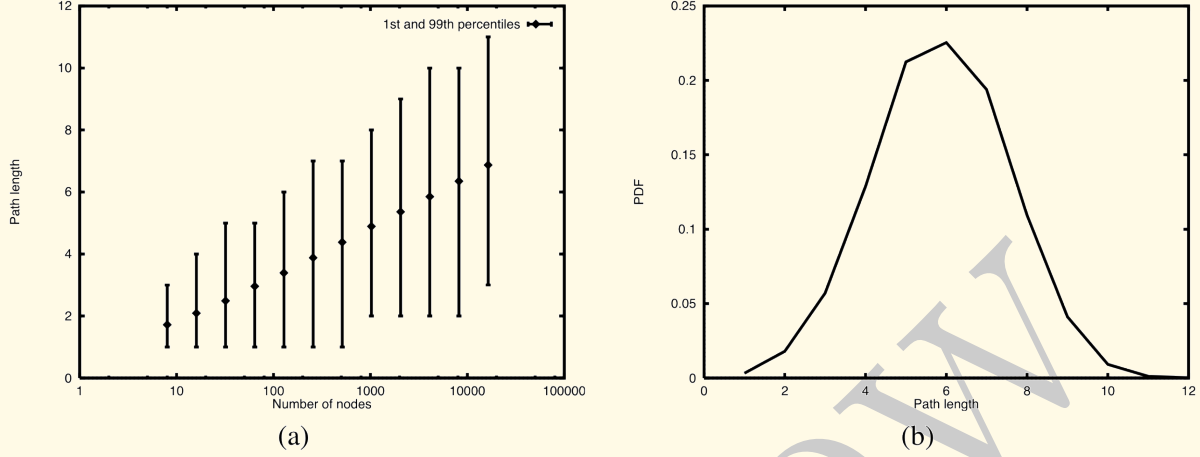


Figure 8: Chord lookup path [7]

Fraction of failed nodes	Mean routing hops	Mean num. of lookup timeouts
0	3.84	0.0
0.1	4.03	0.60
0.2	4.22	1.17

Table 1: Latency on failed node rate of Chord [7]

with its own transport and application workloads before making deployment-level latency claims.

7.2 Reliability

Rings Network allows browser nodes to participate directly in the overlay, which increases expected join and leave rates. Hence, the network’s performance with frequent node churn is crucial for evaluation. Browser nodes may have equal protocol authority without identical persistence, socket, or background-execution capabilities.

Table 2 shows Chord reference simulation results under increasing join/leave rates. Rings should treat these numbers as theoretical guidance and produce independent measurements for the full protocol stack before claiming a concrete availability percentage.

join/leave rate (per sec./stab. period)	Mean routing hops	Mean num. of lookup timeouts	Lookup failures per 10k
0.05/1.5	3.90	0.5	0
0.10/2	3.83	0.11	0
0.15/4.5	3.84	0.16	2
0.20/6	3.81	0.23	5
0.25/7.5	3.83	0.30	6
0.30/9	3.91	0.34	8
0.35/10.5	3.94	0.42	16
0.40/12	4.06	0.46	15

Table 2: Failure rate of Chord [7]

8 Security

As a decentralized platform, Rings Network is susceptible to various security risks and malicious attacks. Therefore, it is crucial to implement robust security measures to ensure the integrity and confidentiality of the network.

- **Sybil Attack:** A Sybil attack represents a malicious attempt to undermine the trust and control of a network by creating multiple fake identities. The target Ranking protocol mitigates this

through admission cost, signed evidence, freeze periods, challengeable claims, and local/global routing signals. Ranking reduces Sybil impact; it does not eliminate Sybil attacks without an economic or credential cost model.

- **Replay Attack:** A replay attack constitutes a malicious attempt to disrupt a network by intercepting and retransmitting a legitimate network transmission. Rings uses digital signatures, cryptographic hashes, timestamps, TTLs, nonces, session expiration, and application-level sequence numbers to limit replay. A Sidecar Sequencer can strengthen replay prevention for critical ordered messages, but only within the finality and comparison rules of the selected witness.
- **Man-in-the-Middle Attack:** A man-in-the-middle (MITM) attack represents a malicious attempt to intercept and potentially modify or eavesdrop on communication between two parties. Rings mitigates MITM attacks by validating signatures, binding public keys to DIDs, signing message envelopes, and encrypting end-to-end payloads. Metadata and traffic-analysis risks remain separate concerns.
- **Denial of Service Attack** A denial-of-service (DoS) attack is a type of cyberattack where an attacker sends a large volume of requests to a network to overwhelm it and disrupt its normal functioning. Rings mitigates DoS through rate limits, bounded chunk reassembly, storage admission policy, ranking, relay quotas, and application-specific costs. These mechanisms bound damage rather than guaranteeing universal availability.

9 Conclusion

This paper presents Rings Network as a target decentralized, peer-to-peer architecture for the digital era of sovereignty. In the ideal model, Rings combines WebRTC, WebAssembly, DID proofs, Chord-derived lookup, signed sessions, encrypted messages, and vir-

tual DIDs to support secure communication, data placement, and service discovery. The remaining engineering task is to validate these protocol claims with implementation-specific measurements, failure models, and extension-runtime constraints.

References

- [1] J. Kshetri. Data privacy and security risks of cloud computing for consumers. *International Journal of Information Management*, 34(6):607–615, Dec 2014.
- [2] E. Kokkonen and A. Porras. Data privacy and security in cloud computing: a review of the state-of-the-art. *Computer Science Review*, 28:21–38, Oct 2018.
- [3] M. R. Grimaila. A comparison of data privacy regulations: the eu and the us. *Journal of International Commerce, Economics and Policy*, 7(3):1–27, 2016.
- [4] Tim Berners-Lee. Internet is dying. [Online; accessed 5-Feb-2023].
- [5] Internet Engineering Task Force. WebRTC: Real-Time Communications between Browsers, 2021. Work in Progress.
- [6] WebAssembly Community Group. WebAssembly, 2017.
- [7] I. Stoica, R. Morris, and D. Karger. Chord: A scalable peer-to-peer lookup service for internet applications. <https://dl.acm.org/doi/10.1145/383059.383071>, 2001.
- [8] Ian Foster and Carl Kesselman. *The Grid: Blueprint for a New Computing Infrastructure*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 2 edition, 2004.
- [9] Edonkey Development Team. Edonkey: A decentralized file sharing system. *International Journal of Distributed Systems*, 5(4):280–290, 2003.

- [10] Gavin Wood. A next-generation smart contract and decentralized application platform, 2014.
- [11] Juan Benet. Interplanetary file system: A p2p file system for the next web. *Proceedings of the 24th International Conference on World Wide Web*, pages 1149–1160, 2015.
- [12] J. R. Etheridge, M. Castro, and I. Stoica. Libp2p: A modular network stack for p2p systems. *USENIX Association Conference on Networked Systems Design and Implementation*, 14:1–15, 2017.
- [13] Petar Maymounkov and David Mazières. Kademlia: A peer-to-peer information system based on the xor metric. *International Workshop on Peer-to-Peer Systems*, pages 53–65, 2002.
- [14] W. Richard Stevens and Gary R. Wright. *TCP/IP Illustrated, Volume 1: The Protocols*. Addison-Wesley Professional, Reading, MA, USA, 1994.
- [15] Carl Hewitt. A universal modular actor formalism for artificial intelligence. *IEEE Transactions on Systems, Man, and Cybernetics*, 3(1):70–80, 1973.
- [16] Pamela Zave. Reasoning about identifier spaces: How to make chord correct. *IEEE Transactions on Software Engineering*, 43(12):1144–1156, 2017. <https://arxiv.org/abs/1610.01140>.
- [17] Saunders Mac Lane. *Categories for the Working Mathematician*. Springer, 1971.
- [18] Eugenio Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991.